

Software Quality Assurance

Assignment 3 — Complete Study Notes

Roll No: 2022F-BCS-022

Subject: Software Quality Assurance (SQA)

Topics: CFG · HFG · Statement Coverage · Decision Coverage

Topics Covered in These Notes

- (a) Control Flow Graph (CFG)
- (b) Hybrid Flow Graph (HFG)
- (c) Statement Coverage
- (d) Decision Coverage
- (e) Achieving 100% Statement Coverage
- (f) Achieving 100% Decision Coverage
- (g) Decision Coverage by Roll Number
- (h) Statement Coverage by Roll Number

Pseudo Code (Roll No ends with 22)

The input is Roll Number. Since it ends with **22**: $A = 22 \times (-1)$, $B = 22 \times (-10)$, $C = 22 \times 50$. Lines 10–13 use a **WHILE loop** (pre-test). Lines 20–23 use a **DO-WHILE loop** (post-test).

Line	Statement
1	Read 22
2	$A = 22 * -1$

3	B = 22 * -10
4	C = 22 * 50
5	X = C + A - B
6	Y = C + B - A
7	Z = A + B + C
8	IF X > Y THEN
9	IF X > Z THEN
10	I = 0
11	WHILE I < 10 DO
12	Print A
13	END WHILE
14	Print B
15	ELSE
16	Print C
17	END IF
18	ELSE
19	IF Y > Z THEN
20	I = 0
21	DO
22	Print A
23	WHILE I < 10
24	Print B
25	ELSE
26	Print C
27	END IF
28	END IF

(a) Control Flow Graph (CFG)

A **Control Flow Graph (CFG)** is a graphical representation of the flow of control through a program. It focuses *only* on decision points — the statements that cause the execution path to branch.

Step 1 — What to include in a CFG?

- Keep all **decision statements** (IF, ELSE, END IF, WHILE, END WHILE, DO, WHILE of DO-WHILE).
- Drop all **non-decision executable statements** (assignments, prints, reads).
- Drop all **non-executable statements** (comments, #include, declarations).
- *Lines 1–7 in this program are all non-decision → they are completely removed from the CFG.*

Step 2 — Decision Statements Identified (this program)

Line	Statement	Type	Notes
8	IF X > Y THEN	IF (outer)	First decision. True → left, False → right
9	IF X > Z THEN	IF (inner)	Inside true-branch of line 8
11	WHILE I < 10 DO	WHILE (pre-test)	Condition tested BEFORE loop body
13	END WHILE	WHILE	Loop back to line 11 if true; exit if false
15	ELSE	ELSE	False branch of inner IF (line 9)
17	END IF	END IF	Closes inner IF (line 9)
18	ELSE	ELSE	False branch of outer IF (line 8)
19	IF Y > Z THEN	IF (inner)	Inside false-branch of line 8
21	DO	DO-WHILE	Post-test: body executes at least once
23	WHILE I < 10	DO-WHILE	Condition tested AFTER loop body
25	ELSE	ELSE	False branch of IF at line 19
27	END IF	END IF	Closes inner IF (line 19)
28	END IF	END IF	Closes outer IF (line 8)

Step 3 — Drawing Rules (MANDATORY)

- **True always goes LEFT, False always goes RIGHT.** This never changes.
- Each IF must have its END IF placed directly below it.
- For nested IFs, the inner IF sits inside the true or false branch of the outer IF.
- The diagram flows top to bottom. Arrows show direction of execution.

Step 4 — Loop Types in CFG

Loop Type	Keywords	Structure in CFG	Entry / Exit
-----------	----------	------------------	--------------

Pre-test	WHILE, FOR	Condition node at TOP. True → body → back to condition. False → exits loop.	Condition checked BEFORE executing body. Body may never execute if condition starts false.
Post-test	DO WHILE, REPEAT UNTIL	DO node first (entry). Body executes. Condition node at BOTTOM. True → back to DO. False → exit.	Body executes at LEAST once regardless of condition.

(b) Hybrid Flow Graph (HFG)

An **HFG** is an extended version of the CFG. Unlike CFG which only shows decision statements, HFG includes **all executable statements**. Non-executable statements (like `#include`, declarations, comments) are the only things removed.

Key Difference: CFG vs HFG

	CFG	HFG
Includes	Decision statements only	All executable statements
Excludes	Non-decision + non-executable	Non-executable only (e.g., <code>#include</code>)
Non-decision execs	Dropped completely	Kept and grouped together
Result	Sparse graph, only branch points	Denser graph with statement groups

Grouping Rules

- Find all consecutive non-decision executable statements.
- Group them into a single numbered node (Group 1, Group 2, ...).
- A group can have 1 statement or 50 — what matters is they are uninterrupted by a decision statement.
- Decision statements (IF, WHILE, etc.) appear as individual nodes just like in CFG.

Groups Identified for This Assignment

Group	Line(s)	Statements Included	Count
①	1–7	READ 22, A=22×−1, B=22×−10, C=22×50, X=C+A−B, Y=C+B−A, Z=A+B+C	7 statements

②	10	I = 0	1 statement
③	12	PRINT A	1 statement
④	14	PRINT B	1 statement
⑤	16	PRINT C	1 statement
⑥	20	I = 0	1 statement
⑦	22	PRINT A	1 statement
⑧	24	PRINT B	1 statement
⑨	26	PRINT C	1 statement

■ In the HFG diagram, wherever a group of statements appeared in the code, you replace them with their group number node. Decision statements remain as individual nodes.

(c) Statement Coverage

Statement Coverage measures what percentage of all statements in a program are executed when a particular test input is given. The goal is 100%, but it is not always achievable.

Formula:

$$\text{Statement Coverage} = \left(\frac{\text{No. of statements executed}}{\text{Total no. of statements}} \right) \times 100$$

Total statements in this program = 28

Dry Run for Input = 22

Roll No = 22 → A = -22, B = -220, C = 1100. Calculating: X = C+A-B = 1100+(-22)-(-220) = 1298, Y = C+B-A = 1100+(-220)-(-22) = 902, Z = A+B+C = (-22)+(-220)+1100 = 858. Since X(1298) > Y(902) → outer IF is **True**. X(1298) > Z(858) → inner IF is **True**. Path = **TT**.

Lines Executed	Count	Statement Coverage
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 28	15 lines	$15/28 \times 100 \approx 57.14\%$

Dry Run for Input = -22

Roll No = -22. X < Y → outer IF is **False**. Execution goes to ELSE (line 18). Y > Z evaluates → inner IF is **False**. Path = **FF**.

Lines Executed	Count	Statement Coverage
1, 2, 3, 4, 5, 6, 7, 8, 18, 19, 25, 26, 27, 28	14 lines	$14/28 \times 100 = 50\%$

Summary Table — Statement Coverage

Input (Roll No)	Path	Lines Executed	Stmts Executed	Statement Coverage
22	TT	1–10, 11, 12, 13, 14, 28	16	57.14%
–22	FF	1–8, 18, 19, 25, 26, 27, 28	14	50.00%

(d) Decision Coverage

Decision Coverage (also called Branch Coverage) measures what percentage of the possible decision paths (branches) through the program are exercised by a test input.

Formula:

$$\text{Decision Coverage} = (\text{No. of decision paths executed} \div \text{Total decision paths}) \times 100$$

How many paths exist in this program?

There are **2 nested IFs**. Each IF has 2 outcomes (T/F). So total paths = $2 \times 2 = 4$ paths.

Path	Outer IF (X > Y)	Inner IF	Result	Triggering Input
TT	True	IF X>Z → True	WHILE loop runs	22
TF	True	IF X>Z → False	ELSE of inner IF	Not possible in this scenario
FT	False	IF Y>Z → True	DO-WHILE runs	Not possible in this scenario
FF	False	IF Y>Z → False	ELSE of inner IF	–22

Decision Coverage per Test Input

Test Input	Paths Executed	Decision Coverage	Explanation
22	TT (1 path)	$1/4 = 25\%$	Only the TT path is triggered
–22	FF (1 path)	$1/4 = 25\%$	Only the FF path is triggered
22 + (–22)	TT + FF (2 paths)	$2/4 = 50\%$	Best achievable with 2 inputs

■ TF and FT paths can never be executed in this scenario because X, Y, Z are all determined by the fixed roll number. Once the input is set, only one combination of T/F is possible for the outer and inner IFs.

(e) 100% Statement Coverage

To achieve 100% statement coverage, every single statement in the program must be executed by at least one test case.

- Maximum statement coverage achievable with 2 test cases (22 and -22) = **75%** (approx, combining both runs).
- Achieving **100% is logically impossible** in this scenario.

Why 100% is impossible:

The values of X, Y, and Z are computed directly from the roll number. Because of the mathematical relationships (e.g., $X = C + A - B$, $Y = C + B - A$), the outer IF ($X > Y$) and inner IF ($X > Z$ or $Y > Z$) can only take certain combinations of T/F for any given input. The TF and FT paths lead to lines that can never be reached regardless of what value you input.

(f) 100% Decision Coverage

To achieve 100% decision coverage, all 4 decision paths (TT, TF, FT, FF) must be executed.

- With 2 test cases: maximum = **50%** (TT + FF only).
- 100% decision coverage is **not achievable** in this scenario for the same reason as above.
- If 100% were achievable, you would need at least 4 different test inputs to cover TT, TF, FT, FF.

(g) Decision Coverage by Roll Number

Last 3 digits of roll number = **022**. Using 022 as input:

- Roll No = 22 → executes the **TT path**.
- 1 out of 4 decision paths is executed.

$$\text{Decision Coverage} = 1/4 \times 100 = 25\%$$

(h) Statement Coverage by Roll Number

Using roll number **022** as input → **57.14% statement coverage**.

Lines that WILL execute:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 28

Lines that will NOT execute:

15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27 (12 lines)

These 12 lines are in the ELSE branches and DO-WHILE section which are unreachable when the outer IF ($X > Y$) is True (which it is for input 22).

Quick Reference Summary

Concept	Key Rule	This Assignment
CFG	Keep decision stmts only (IF, WHILE, ELSE, END IF, END WHILE, DO). Drop all non-decision lines.	Lines 1–7 dropped. 13 decision nodes remain.
HFG	Keep all executable stmts. Group consecutive non-decision ones. Drop only non-executable.	9 groups identified. Decision nodes as individual nodes.
Stmt Coverage	$(\text{Stmts executed} \div \text{Total stmts}) \times 100$ Total = 28 stmts.	Input 22 → 57.14% Input -22 → 50% Max = 57.14%
Decision Coverage	$(\text{Paths executed} \div \text{Total paths}) \times 100$ Total = 4 paths (TT,TF,FT,FF).	Input 22 → 25% (TT) Input -22 → 25% (FF) Max = 50%
100% Coverage	Try test inputs: +ve, -ve, zero, boundary values.	Not achievable in this scenario. TF and FT paths are unreachable.
Roll No 022	Use as test input, trace which path executes.	Stmt: 57.14% Decision: 25% (TT path)